

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчёт о научно-исследовательской работе

магистранта

Семестр 2

Направление подготовки: 01.04.02 Прикладная математика и информатика:

Магистерская программа «Системы искусственного интеллекта»

Профиль Системы искусственного интеллекта

Студент Васильев Артём Сергеевич

группы 6131 – 010402D

Тема научно-исследовательской работы: Моделирование фокального сдвига системы гармонических линз в видимом диапазоне методом трассировки лучей.

Научный руководитель
цент, НИЛ-35

Скиданов Р.В. профессор кафедры киберфотоники, д.ф-м.н, до-

Научный руководитель

(подпись)
“ ” _____ 2026 г.

Студент

(подпись)
“ ” _____ 2026 г.

Самара 2027

РЕФЕРАТ

Отчет по научно-исследовательской работе: 41 с., 17 рисунков, 3 таблицы, 16 источников, 1 приложение.

ТРАССИРОВКА ЛУЧЕЙ, ГАРМОНИЧЕСКАЯ ЛИНЗА, ФОКАЛЬНЫЙ СДВИГ, ХРОМАТИЧЕСКИЕ АБЕРРАЦИИ, СФЕРИЧЕСКИЕ АБЕРРАЦИИ, ОПТИМИЗАЦИЯ

Объект исследования: Системы из двух, трех и четырех гармонических дифракционных линз (ГДЛ) в видимом диапазоне излучения.

Цель работы: Верификация алгоритма определения оптимальных высот микрорельефа для минимизации фокального отрезка и сравнительный анализ параксиального приближения с методом реальной трассировки лучей.

В ходе работы был разработан комплексный метод оптимизации и специализированное программное обеспечение на языке Python, реализующее алгоритмы трассировки лучей с учетом сферических aberrаций и дисперсии материала ВК7.

Проведены вычислительные эксперименты, которые показали, что параксиальное приближение обладает существенной неточностью, занижая реальную величину фокального сдвига примерно в 3 раза во всех исследованных конфигурациях.

Подтверждена адекватность разработанного алгоритма на системе из 2-х линз, где результаты совпали с теоретическими данными.

Для сложных систем получены количественные результаты оптимизации: использование метода трассировки позволило сократить длину фокального отрезка в 2.6 раза для трехлинзовой системы и в 3.2 раза для системы из четырех линз.

Доказано, что применение реальной трассировки позволяет уменьшить фокальное пятно на величину до 3.44 мм, что критически важно для проектирования прецизионных оптических систем.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Теоретические основы и математический аппарат	7
2 Алгоритмы трассировки лучей	11
2.1 Многомерный метод бисекции.....	11
2.2 Расчёт фокального отрезка матричным методом	11
2.3 Общий алгоритм трассировки	12
3 Разработка программного обеспечения	14
3.1 Класс для работы с трассировкой	14
3.2 Класс для работы с изображением.....	14
3.3 Класс для работы с графиками	14
3.4 Класс для работы с параксиальной оптимизацией.....	15
3.5 Второй класс для работы с трассировкой.....	15
4 Моделирование системы гармонических линз.....	17
4.1 Параксиальный случай.....	17
4.1.1 Опыт с двумя гармоническими линзами	17
4.1.2 Опыт с тремя гармоническими линзами.....	19
4.1.3 Опыт с четырьмя гармоническими линзами	21
4.2 Трассировка.....	22
4.2.1 Опыт с двумя гармоническими линзами	22
4.2.2 Опыт с тремя гармоническими линзами.....	24
4.2.3 Опыт с четырьмя гармоническими линзами	27
ЗАКЛЮЧЕНИЕ.....	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А Базовый класс для трассировки лучей.....	32

ВВЕДЕНИЕ

Гармоническая дифракционная линза (ГДЛ) — это гибридное устройство, в котором интегрированы принципы рефракционной и дифракционной оптики. Ключевая особенность такого решения состоит в компенсации хроматических aberrаций за счет взаимного подавления разнонаправленного хроматизма обоих компонентов [4]. В то время как классические рефракционные линзы обладают избыточной толщиной, затрудняющей миниатюризацию приборов, дифракционные элементы (ДЛ) имеют микрорельеф порядка длины волны λ_0 , обеспечивая компактность. Тем не менее, узкая специализация ДЛ на конкретной рабочей длине волны обуславливает высокий уровень хроматических искажений, который в ГДЛ нивелируется интеграционным подходом.

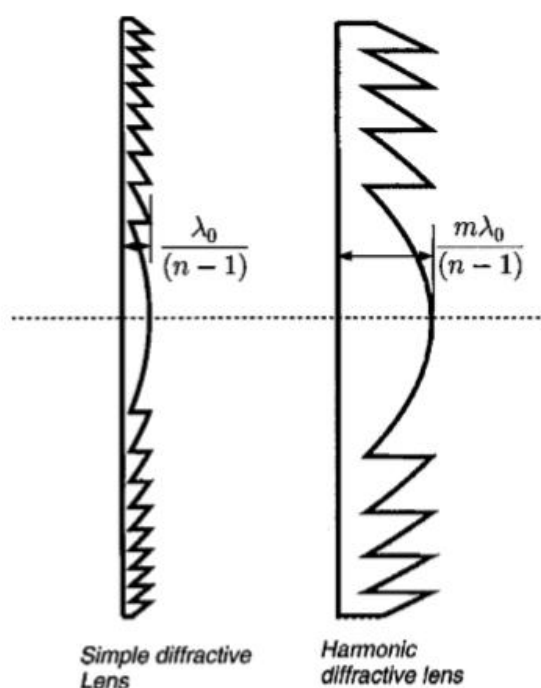


Рисунок 1 – Сравнение дифракционной с гармонической линзой

Гармонические линзы демонстрируют уменьшенную хроматическую aberrацию и способны фокусировать в одной точке несколько длин волн благодаря работе в различных порядках дифракции. Однако «рабочие длины волн» ГДЛ не являются произвольными и определяются аналитическим выражением, зависящим от гармоники M и базовой длины волны λ_0 [4-5]. Даже при незначительных отклонениях (порядка нескольких процентов) от расчётных длин волн наблюдается существенная деградация качества фокального пятна, что представляет собой серьёзный недостаток данной технологии.

Понятие «гармоническая линза» относится к дифракционному элементу с пилообразным фазовым микрорельефом, функционирующему в высоких порядках дифракции ($k \gg 1$). Глубина рельефа H такой линзы существенно превышает расчётную длину волны λ_0 [8-10]. Апертура линзы разделена на гармонические зоны Френеля, где разность расстояний

от границ каждой зоны до фокальной точки кратна произведению порядка гармоничности m на расчётную длину волны [7].

Проблема оптимизации рабочих характеристик гармонических линз для их эксплуатации в специфических условиях остается приоритетной задачей современной прикладной оптики. Инженерное проектирование таких элементов неразрывно связано с поиском баланса между совокупностью критических параметров: геометрией микрорельефа, базовой оптической силой и дисперсионными свойствами материала. Высокая чувствительность фокусирующих свойств ГДЛ к вариации любого из указанных факторов обуславливает необходимость разработки прецизионных вычислительных методов.

Сложность синтеза подобных систем проистекает из нелинейной многомерной зависимости выходных оптических характеристик от конструктивных переменных. В связи с этим актуализируется потребность в специализированном алгоритмическом обеспечении, способном эффективно минимизировать фокальный сдвиг или достигать иных целевых критериев фокусировки. В рамках данной работы для решения задач анализа и оптимизации фокального отрезка применяется метод численной трассировки лучей.

Являясь фундаментальным инструментом геометрической оптики, трассировка лучей (ray tracing) базируется на математическом моделировании траекторий распространения излучения. Теоретическим фундаментом метода служит принцип Ферма, однако в современных вычислительных реализациях процесс опирается на итерационный расчет актов преломления и отражения на границах раздела сред. В отличие от ресурсозатратных методов волновой оптики, требующих решения уравнений Максвелла, данный подход использует аппроксимацию светового луча как линии, нормальной к волновому фронту в пределе $\lambda \rightarrow 0$. Такое допущение обеспечивает высокую производительность вычислений для систем, чьи габариты многократно превосходят длину волны, при этом современные расширения алгоритмов позволяют интегрировать фазовые характеристики для учета дифракционных явлений.

Особую значимость метод трассировки приобретает при проектировании высокосложных элементов, таких как гармонические линзы, характеризующиеся сильной дисперсией и специфическим профилем поверхности. Инструментарий метода позволяет не только проводить комплексный анализ aberrаций и оценивать качество формируемого изображения, но и исследовать влияние производственных допусков на итоговую эффективность системы.

Математическая реализация используемого алгоритма включает векторное описание лучей и решение уравнений их пересечения с поверхностями различной топологии. В

случаях, когда аналитическое описание границ раздела невозможно, применяются численные методы (например, итерации Ньютона — Рафсона). Для обеспечения адекватности модели также выполняется интеграция данных о дисперсии материалов и поляризационных эффектах, что позволяет с высокой точностью прогнозировать поведение излучения в структурах с глубоким микрорельефом.

Целью исследования является верификация алгоритма определения оптимальных высот, обеспечивающих минимизацию фокусного расстояния (в параксиальном приближении), применительно к расчету реальной трассировки лучей. Реализация алгоритма предусмотрена для оптических систем, включающих две, три и четыре гармонические линзы. В работе проводится оценка погрешности предложенного метода для параксиального случая в данных конфигурация.

1 Теоретические основы и математический аппарат

Трассировка лучей является детерминированным вычислительным методом, основанным на принципах геометрической оптики, и моделирует распространение электромагнитного излучения в средах с пространственно-неоднородными оптическими свойствами. Метод базируется на концепции световых лучей — математических абстракций, описывающих направление распространения энергии в приближении нулевой длины волны ($\lambda \rightarrow 0$).

Для применения данного подхода необходимо определить границы его применимости:

$$\lambda \ll R, \quad (1.1)$$

где R — радиус кривизны волнового фронта.

$$\lambda \ll l_c, \quad (1.2)$$

где l_c — масштаб неоднородностей.

Далее излагается математический аппарат, используемый в работе.

Изначально необходимо задать модель луча в пространстве, который описывается параметрическими уравнениями:

$$\begin{cases} x(t) = x_0 + a_x \cdot t \\ y(t) = y_0 + a_y \cdot t \\ z(t) = z_0 + a_z \cdot t \end{cases}, \quad (1.3)$$

где (x_0, y_0, z_0) — начальная точка луча;

(a_x, a_y, a_z) — направляющий вектор луча;

t — параметр вдоль луча.

Представим поверхности линзы дискретно с помощью следующих формул:

$$r_i = D/2N \cdot i, i = 0..N - 1; \quad (1.4)$$

$$z(r_i) = z_{lens} - S(r_i), \quad (1.5)$$

где D — диаметр линзы;

N — кол-во точек дискретизации;

$S(r_i)$ — функция профиля линзы;

z_{lens} — положения центра линзы по оси z .

Запишем уравнения поверхностей разных видов линз.

Сферическая поверхность представляется таким образом:

$$S(r) = T + k * \sqrt{(R^2 - r^2)} - R, \quad (1.6)$$

где $R = P$;

T – толщина линзы;

$k = \text{sign}(R)$ – направление кривизны.

Параболическая поверхность:

$$S(r) = T - r^2/(2P), \quad (1.7)$$

где P – параболический параметр (фокусное расстояние параболоида).

Аксикон:

$$S(r) = T - r/P, \quad (1.8)$$

где P – параметр аксикона.

Обобщенная параболическая поверхность:

$$S(r) = T - r^n/(2P), \quad (1.9)$$

где n – степень обобщения.

Кроме того, требуется описать поведение лучей на границе раздела сред.

$$\cos(\theta_1) = a \cdot N = a_x N_x + a_y N_y + a_z N_z; \quad (1.10)$$

$$\sin(\theta_1) = \sqrt{(1 - \cos^2(\theta_1))}; \quad (1.11)$$

$$\sin(\theta_2) = (n_1/n_2) \cdot \sin(\theta_1), \quad (1.12)$$

где θ_1 – угол падения;

θ_2 – угол преломления;

n_1, n_2 – показатели преломления сред.

В качестве моделируемого объекта рассматривается многолинзовая система. Чтобы получить ожидаемый результат необходимо использовать матрицы преломления и переноса в паре.

Данные матрицы имеют размер 2×2 , но нужна лишь конечная матрица в ходе их перемножения. Матрица переноса и матрица преломления рассчитываются следующим образом:

$$T_n = \begin{pmatrix} 1 & \frac{d_n}{n} \\ 0 & 1 \end{pmatrix}, \quad (1.13)$$

$$R = \begin{pmatrix} 1 & 0 \\ -\Phi_i & 1 \end{pmatrix}, \quad (1.14)$$

где T_n – матрица переноса;

R – матрица преломления;

Матрица такой системы будет состоять из произведения матриц преломления и матриц переноса для отдельных участков системы. В результате получится набор матриц, которые нужно перемножить в обратном порядке. Обозначим итоговую матрицу G ,

$$G = T_3 R_2 T_2 R_1 T_1 T_0 = T_n R_n \dots T_1 R_1 T_0, \quad (1.15)$$

которая будет иметь размерность 2×2 с элементами A , B , C и D .

$$G = \begin{bmatrix} A & B \\ C & D \end{bmatrix}. \quad (1.16)$$

В конечном счёте произведения матриц получится матрица вида (1.14), в которой необходим лишь элемент C , который связан с оптической силой простым уравнением:

$$C = -\Phi. \quad (1.17)$$

Таким образом, элемент C равен оптической силе, взятой со знаком минус.

Свяжем фокусное расстояние и оптическую силу:

$$f = \frac{1}{D}. \quad (1.18)$$

В данном исследовании также будет необходимо ввести понятия и формулы для матричного метода.

Суть данного метода заключается в перемножении матриц переноса и матриц преломления для оптической системы.

Помимо матриц, необходимо задать фокус оптической системы:

$$f(\lambda) = \frac{m\lambda_0}{k\lambda} f_0, \quad (1.19)$$

где m – гармоника линзы;

λ_0 – базовая длина волны;

λ – длина волны падающего излучения;

f_0 – базовый фокус линзы;

k – параметр, который рассчитывается по формуле $k = \text{round}\left(\frac{m\lambda_0}{\lambda}\right)$, где оператор *round* играет роль функции округления до ближайшей гармоники.

2 Алгоритмы трассировки лучей

2.1 Многомерный метод бисекции

Для нахождения точки пересечения с линзой применён метод бисекции. Для многоэлементной оптической системы метод бисекции можно обобщить как последовательный итерационный метод поиска траектории луча через систему, где на каждом оптическом элементе решается задача пересечения с поверхностью. Это превращает одномерный метод поиска корня в многомерный итерационный процесс с адаптивными границами поиска.

Что касается метода трассировки лучей, то данный алгоритм выглядит следующим образом:

Для каждого луча с начальными параметрами $(x_0, y_0, z_0, a_x, a_y, a_z)$,

где x_0 – координата луча по горизонтали;

y_0 – координата луча по вертикали;

z_0 – координата луча вдоль оптической оси;

a_x, a_y, a_z – направляющие векторы луча.

Для каждой линзы $k = 1 \dots N$ (N – число линз):

Если луч достигим для линзы k :

1. Найти точку пересечения с поверхностью k методом бисекции;
2. Вычислить преломленный/отраженный луч;
3. Обновить параметры луча.

Иначе:

1. Завершить трассировку для этого луча.

2.2 Расчёт фокального отрезка матричным методом

Также в данной работе разработан алгоритм нахождения глобального минимума для системы гармонических линз в параксиальном приближении.

Чтобы перейти к реализации алгоритма, необходимо задать целевую функцию. В данном случае это функция определения длины фокального отрезка для оптической системы, которая будет проходиться по выбранному диапазону длин волн и на каждой итерации считать фокус системы, используя матричные методы, которые были описаны ранее в теоретическом разделе. После того как алгоритм прошелся по всем длинам волн и построил список всех фокусов $array_f$, необходимо из этого списка найти фокальный отрезок следующим образом:

$$\Delta_f = \max(array_f) - \min(array_f), \quad (2.1)$$

где $\max()$, $\min()$ – операторы нахождения максимума и минимума для массива (списка).

Чтобы реализовать алгоритм необходимо использовать матричные методы оптики и параксиальное приближение.

В представленной методологии реализуется пошаговый расчет фокусных расстояний для дискретного набора длин волн с учётом дисперсионных свойств оптических элементов. Алгоритм включает следующие основные этапы:

1. На первом этапе генерируется упорядоченный набор длин волн в заданном интервале $[\lambda_{min}, \lambda_{max}]$. Шаг дискретизации h_λ выбирается исходя из требуемой точности расчётов; в данной реализации использовано значение 0.5 нм;
2. Для каждой длины волны λ_i из сформированного списка последовательно выполняются следующие операции:
 - 2.1 Построение матрицы системы: на основе параметров каждого оптического элемента (линзы или промежутка) формируются матрицы преломления и переноса согласно формулам (1.13) и (1.14). Для каждого компонента системы в матрицу заносятся соответствующие значения показателя преломления среды и геометрических расстояний;
 - 2.2 Вычисление результирующей матрицы: путём перемножения матриц всех компонентов системы в порядке их расположения получается общая матрица преобразования (1.16);
 - 2.3 Определение фокусного расстояния: из результирующей матрицы извлекается элемент C , связанный с фокусным расстоянием f соотношением (1.17). Фокусное расстояние для текущей длины волны вычисляется по формуле (1.19).

Таким образом, алгоритм реализует последовательный спектральный анализ оптической системы, комбинируя матричные методы параксиальной оптики с учётом дисперсии материалов. Ключевыми особенностями метода являются использование дискретного спектрального представления, итеративное применение матричного формализма для каждого оптического компонента и последующее обобщение результатов в виде интегрального критерия - фокального отрезка.

2.3 Общий алгоритм трассировки

Трассировка основана на методе лучевых матриц, дополненном нелинейной поправкой для моделирования аббераций. Процесс идет последовательно от первой линзы к последней.

Первый этап включает в себя инициализацию луча, а именно:

1. высота луча относительно оптической оси;
2. угол наклона к оси.

Второй этап представляет собой расчёт физических свойств материала (BK7).

Для каждой линзы вычисляется показатель преломления n в зависимости от длины волны λ по формуле Зельмейера:

$$n^2(\lambda) = 1 + \sum_{i=1}^3 \frac{B_i \lambda^2}{\lambda^2 - C_i}.$$

Последний этап - моделирование гармонической линзы.

Поскольку линза «гармоническая», то фокусное расстояние зависит от микрорельефа. Алгоритм вычисляет:

1. базовую длину волны λ_0 , на которую «настроена» высота микрорельефа;
2. порядок дифракции k : ближайшее целое число, показывающее, в какой порядок попадает текущая длина волны;
3. эффективное фокусное расстояние: $f_{eff} = \frac{m\lambda_0}{k\lambda} f_0$.

Внутри цикла *for i in range(1, count_linse + 1)* происходят следующие ключевые действия:

Шаг 1: Учет сферической аберрации.

Вводится коэффициент $SA3$ (сферическая аберрация 3-ого порядка).

Вычисляется «эффективная оптическая сила»: $D_{eff} = D_{paraxial} \cdot (1 + SA3 \cdot h^2)$.

Чем дальше луч от центра (h больше), тем сильнее линза его изгибает. Это имитирует реальную физику, где сферические поверхности фокусируют краевые лучи ближе к линзе, чем центральные.

Шаг 2: Изменение угла (преломление).

Угол луча обновляется по формуле:

$$\alpha_{new} = \alpha_{old} - (D_{eff} \cdot h).$$

Это мгновенное изменение направления луча в плоскости линзы.

Шаг 3: Перенос (пролет до следующей линзы).

Если это не последняя линза, луч проходит через свободное пространство или среду до следующей поверхности:

1. z_{pos} увеличивается на дистанцию между линзами;
2. новая высота $h_{new} = h_{old} + \alpha_{new} \cdot dist$;
3. учитывается приведённое расстояние в $dist$, что позволяет моделировать прохождение луча через среды с разным коэффициентом преломления.

3 Разработка программного обеспечения

3.1 Класс для работы с трассировкой

Для трассировки реализован класс *HarmLens*, содержащую логику визуализации и расчёта для многолинзовой системы. Данный класс описывает следующие атрибуты:

N_x, N_y, N_z – компоненты нормали к поверхности;

x_0, y_0, z_0 – начальные координаты луча;

x_L, y_L, z_L – конечные координаты луча;

α, β, γ – направляющие косинусы входного луча;

$\alpha_L, \beta_L, \gamma_L$ – направляющие косинусы выходного луча;

Diameter – диаметр линзы;

Thickness – толщина линзы;

n_1, n_2 – показатели преломления (до и после линзы);

Type – тип линз;

Section – массив значений сечения профиля.

Метод *int ReadFile(name : str)* позволяет считать параметры линзы из файла.

Метод *int P_Lens(l : float, n₁ : float, n₂ : float)* рассчитывает профиль гармонической линзы, где

l – длина волны;

n_1, n_2 – показатели преломления.

Метод *int Harmonization(float h)* создаёт гармонический профиль с высотой микро-рельефа h .

Метод *double BeamAxe()* вычисляет положение фокуса на оптической оси.

3.2 Класс для работы с изображением

Помимо основного класса *HarmLens* реализован вспомогательный класс для работы с изображением *BmpFile*, где

OpenFile(filename : str, w, h, bits) – создаёт новый BMP файл с заданными размерами;

OpenFile(filename : str) – открывает существующий BMP файл;

ReadShape(fileSize) – читает заголовок и параметры изображения;

WriteShape(size) – записывает заголовок BMP;

GetFileSize(w, h, bits) – вычисляет размер файла в байтах.

3.3 Класс для работы с графиками

Класс *GR* предназначен для работы с графиками с методами

ReadGR(name, x, y, N) – чтение графика из файла;

WriteGR(name, x, y, N) – записывает график в файл;

$SpGR(mas, N, l_1, l_2, lam)$ – интерполирует значение на длине волны.

3.4 Класс для работы с параксиальной оптимизацией

Класс *BaseLensOptimizer* реализует параксиальный расчёт для системы гармонических линз. В классе содержатся методы для вычисления фокального сдвига, визуализация зависимости длины волны от фокусного расстояния для произвольного числа линз и визуализация алгоритма оптимизации для 2-х, 3-х и 4-х гармонических линз.

Класс хранит следующие поля:

lmd_f_dict – словарь данных, где ключ – список фокусов, значение – список всех длин волн;

len_f_dist – размер фокального отрезка (сдвига);

dataset – входные данные для расчёта.

Метод *calc_focus_dist(lmbd_f_dict : dict[float, float])* принимает словарь данных, где ключ – список фокусов, значение – список всех длин волн. Разница между максимальным и минимальным значением из списка фокусов есть фокальный сдвиг.

Метод *lmbd_focus_dist(dataset, heights, lambda_massive, return_dict)* принимает словарь *dataset* входных данных, если он не был задан в конструкторе класса; *heights* – список высот, необходимых для оптимизации; *lambda_massive* – дискретизация массива длин волны; *return_dict* – нужно ли возвращать словарь после завершения вызова данного метода.

Метод *visualize_depend_f_lmbd(units, return_fig_ax)* принимает параметр *units* размерность для расчёта (длина волны, фокусное расстояние и фокальный отрезок); *return_fig_ax* – нужно ли возвращать объект *Figure* и *Axis* после отрисовки.

3.5 Второй класс для работы с трассировкой

Класс *RealRayTracer* инициализируется через *__init__(dataset)*, где *dataset* — словарь с параметрами оптической системы. Из него сразу извлекается *count_lins* — количество линз, которое используется во всех методах как счётчик итераций.

Метод *n_bk7(lambda_um)* — вспомогательный метод. Принимает длину волны в микрометрах и возвращает показатель преломления стекла *BK7* по трёхчленной формуле Зельмейера. Используется внутри каждого метода трассировки для расчёта дисперсии.

Метод *trace_ray_real(...)* — центральный метод класса. Принимает начальную высоту луча *h_start*, длину волны *lmbd*, список высот микрорельефа линз *heights* и радиус апертуры. Для каждой линзы последовательно: вычисляет эффективную оптическую силу с поправкой на сферическую aberrацию ($effective_power = optic_power * (1 + SA3 * h^2)$), применяет преломление и перенос до следующей линзы. Возвращает тройку (*h_final*,

alpha_final, trace_history) — финальную высоту, угол и историю координат по всем поверхностям.

Метод *trace_multiple_rays(...)* — обёртка над *trace_ray_real*. Генерирует *num_rays* лучей, равномерно распределённых от оси до края апертуры, трассирует каждый и собирает результаты в словарь с ключами *ray_heights*, *focus_positions*, *transverse_aberration*, *all_traces*.

Метод *find_focus_with_spherical_aberration(...)* — аналитический метод. Трассирует два луча — на 1% и 99% от апертуры — и по ним вычисляет параксиальный фокус, крайвой фокус, продольную сферическую абберацию (их разность) и «лучший фокус» как среднее арифметическое.

4 Моделирования системы гармонических линз

4.1 Параксиальный случай

4.1.1 Опыт с двумя гармоническими линзами

В ходе исследования будет проведен опыт с двумя, тремя и четырьмя линзами, где в качестве параметра оптимизации будет выступать высота микрорельефа.

В данном разделе будет описан алгоритм нахождения оптимальной высоты второй линзы при фиксированной первой или наоборот.

В ходе моделирования должно получиться, что найденная высота, при которой достигается минимальное значение фокального отрезка, должна соответствовать формуле (4.1) из статьи [6]:

$$h_1 = h \frac{m \pm 0.5}{m}, \quad (4.1)$$

где m – гармоника высоты h .

То есть, если зафиксировать h равной 7 мкм при гармонике 7, то высота второй линзы h_1 должно равняться 7.5 мкм при гармонике 7.5 либо 6.5 мкм при гармонике 6.5.

Для 2-х гармонических линз, где одна из линз будет иметь фиксированную высоту и гармонику, алгоритм будет формировать список значений фокальных отрезков и высот для второй линзы.

Алгоритм оптимизации высоты микрорельефа для двухкомпонентной системы гармонических линз.

При работе с оптической системой, состоящей из двух гармонических линз, где параметры первой линзы (высота микрорельефа и номер гармоники) остаются неизменными, реализуется процедура поиска оптимальной высоты второй линзы. Данный алгоритм базируется на систематическом переборе значений высоты в определённом диапазоне с последующей оценкой качества системы через расчёт фокального отрезка.

Последовательность выполнения алгоритма:

Определение исследуемого диапазона

1. Устанавливаются начальное и конечное значения высоты микрорельефа, формирующие интервал для последующего анализа;
2. В рамках заданного интервала генерируется массив значений высоты с равномерным шагом, обеспечивающим требуемую детализацию исследования.

Для каждого значения высоты из сформированного списка выполняется:

1. Расчёт оптических характеристик системы с учётом текущей высоты второй линзы;
2. Определение соответствующего значения фокального отрезка;
3. Полученные результаты сохраняются в виде упорядоченного списка;
4. Среди вычисленных значений фокального отрезка находится минимальное с использованием стандартной функции `min()`. Индекс этого минимального значения в списке определяется посредством функции `numpy.argmin()`. Данный индекс позволяет извлечь из массива высот конкретное значение, при котором достигается наилучший (минимальный) фокальный отрезок.

Таблица 1 – Данные для 2-х гармонических линз

№ линзы	Гармоника	Базовая длина волны, нм	Базовый фокус, мм	Расстояние между линзами, мм
1	7	550	200	10
2	7	550	200	10

Для начального приближения возьмём высоты двух линз, равным 7 мкм. Проверим значение фокального отрезка для диапазона 3 мкм до 7 мкм. На рисунке 2 изображена точка, показывающая при какой высоте достигается минимальное значение фокального пятна. Для расчёта использовались данные из таблицы 1.

Поведение фок.отрезка от изменения высоты | Нач.приближение = 7.0, 7.0

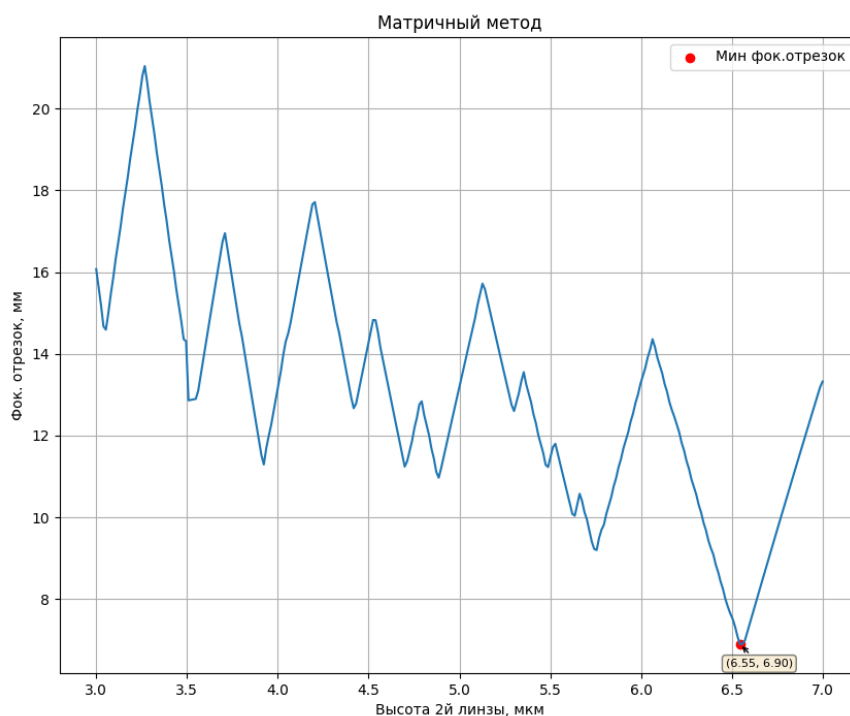


Рисунок 2 – График зависимости фокального отрезка
от высоты линзы при гармонике 6.5

Для гармонике 7.5 алгоритм ищет оптимальную высоту в диапазонах от 7 мкм до 12 мкм. На рисунке 2 можно увидеть найденный минимум при высоте 7.48 мкм.

Поведение фок.отрезка от изменения высоты | Нач.приближение = 7.0, 7.0

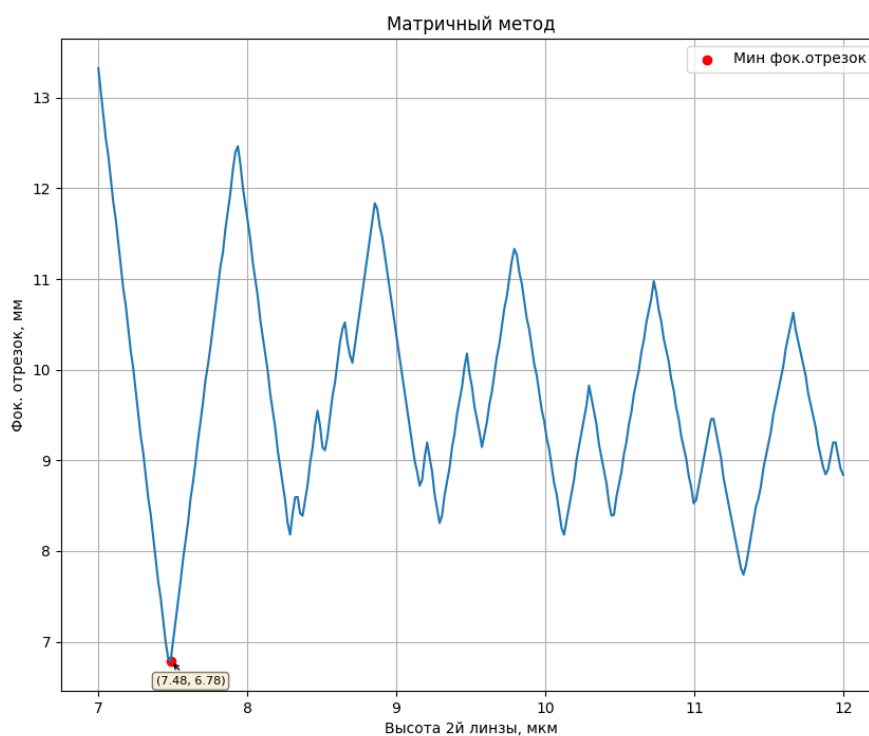


Рисунок 3 – График зависимости фокального отрезка
от высоты линзы при гармонике 7.5

4.1.2 Опыт с тремя гармоническими линзами

Рассмотрим метод определения оптимальных значений высоты микрорельефа для оптической системы, состоящей из трёх гармонических линз. Алгоритм включает последовательность действий:

1. Задаются граничные значения интервала, в пределах которого будет проводиться поиск оптимальных высот микрорельефа.
2. Одна из трёх линз выбирается в качестве элемента с фиксированными параметрами. Для оставшихся двух линз выполняется подбор оптимальных высот.
3. Осуществляется двумерный поиск по значениям высот для двух оптимизируемых линз с использованием вложенных циклов. В процессе расчётов формируется двумерная матрица, размерность которой определяется количеством дискретных значений высот в заданном диапазоне. Каждая ячейка этой матрицы

содержит величину фокального отрезка, вычисленную для соответствующей пары высот.

4. После заполнения матрицы выполняется поиск глобального минимума фокального отрезка. Для этого применяется функция `numpy.argmin()`, возвращающая индекс минимального элемента в плоском представлении матрицы. Затем с помощью функции `numpy.unravel_index()` осуществляется преобразование одномерного индекса в двумерные координаты (номер строки и столбца), соответствующие положению минимального элемента в исходной матрице. Найденные координаты определяют комбинацию высот микрорельефа, при которой достигается минимальное значение фокального отрезка.

Рассмотрим при каких высотах достигается минимальный фокальный отрезок, если фиксировать линзы поочередно с начальной высотой в 7 мкм.

Таблица 2 – Данные для 3-х гармонических линз

№ линзы	Гармоника	Базовая длина волны, нм	Базовый фокус, мм	Расстояние между линзами, мм
1	7	550	300	10
2	7	550	300	10
3	7	550	300	10

На рисунке 4 изображена точка, в которой достигается минимальное значение фокального отрезка при оптимальных высотах. При фиксировании высоты первой линзы в 7 мкм получились высоты в 13.88 мкм, 14.18 мкм для второй и третьей линзы соответственно.

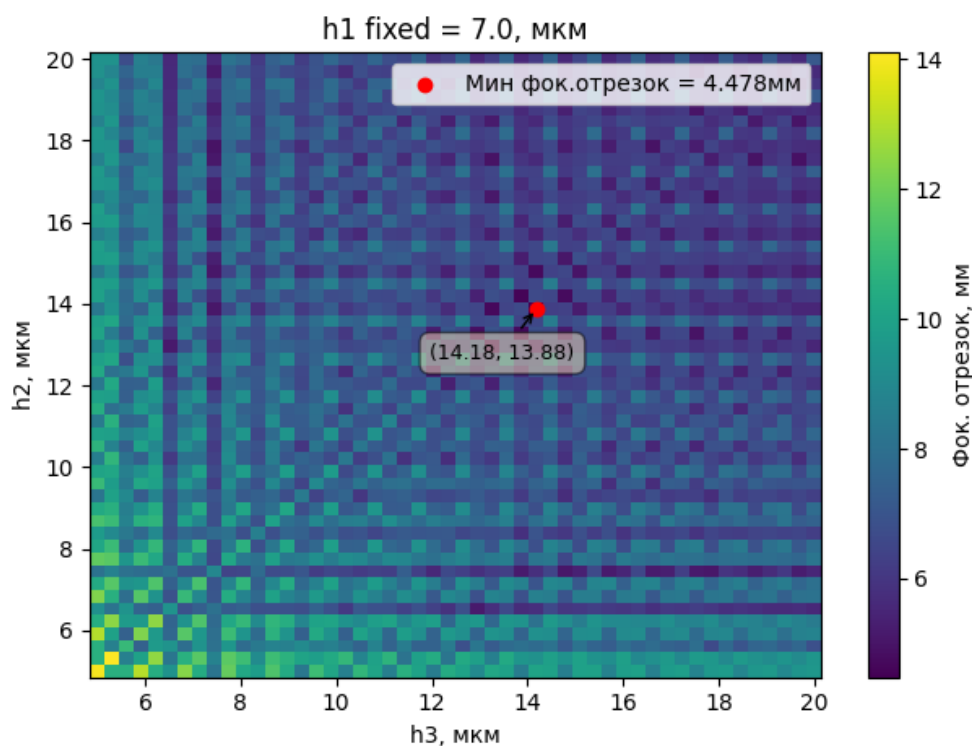


Рисунок 4 – Цветовая схема при фиксировании 1-й линзы

4.1.3 Опыт с четырьмя гармоническими линзами

Рассмотрим заключительный случай – систему, состоящую из четырёх гармонических линз. В ходе исследования фиксируется высота одной из линз, в то время как значения высот остальных трёх подбираются методом полного перебора (Grid Search).

Алгоритм оптимизации в этом случае отличается от алгоритма для трёхлинзовой системы лишь добавлением внешнего цикла, необходимого для перебора значений дополнительной переменной. Высота фиксированной линзы в алгоритме равна 7 мкм.

Таблица 3 – Данные для 4-х гармонических линз

№ линзы	Гармоника	Базовая длина волны, нм	Базовый фокус, мм	Расстояние между линзами, мм
1	7	550	400	10
2	7	550	400	10
3	7	550	400	10
4	7	550	400	10

На рисунке 5 представлена 3D-визуализация процесса оптимизации фокального отрезка для четырёх гармонических линз при фиксированной высоте первой линзы.

Диапазон поиска по каждой высоте составлял от 5 мкм до 15 мкм, объем исследования составил $50 \times 50 \times 50$. Итоговое значение фокального отрезка составило 3.7 мм, что примерно в 3.2 раза меньше, чем в случае одиночной гармонической линзы.

Фокальный отрезок

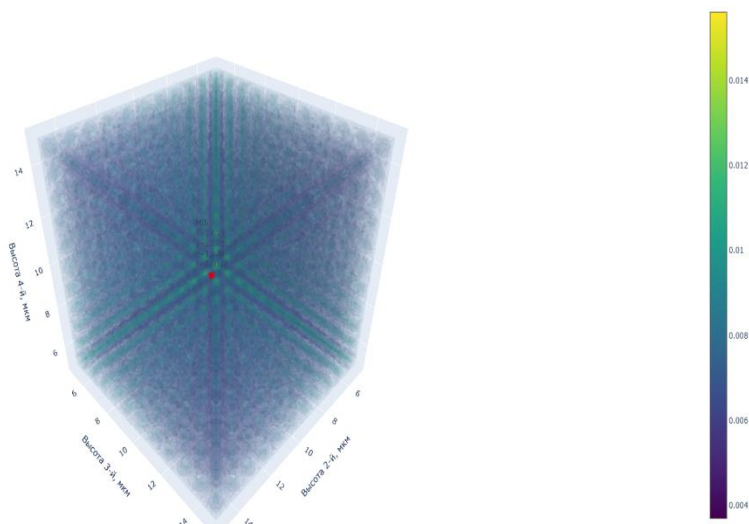


Рисунок 5 – 3D визуализация для 4-х линз при фиксировании 1-й линзы

4.2 Трассировка

В данном разделе представлены результаты вычислительных экспериментов, в которых параксиальное приближение сопоставляется с методом реальной трассировки лучей.

4.2.1 Опыт с двумя гармоническими линзами

Данные для моделирования взяты из таблицы 1.

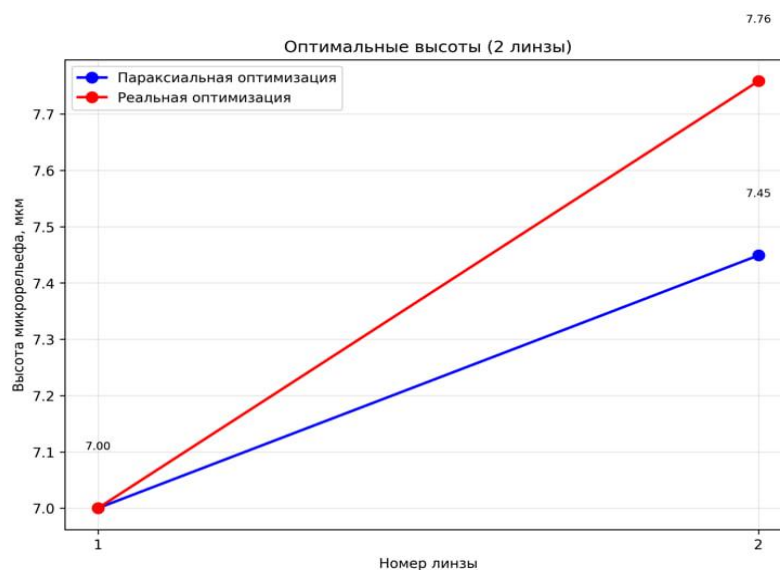


Рисунок 6 – Оптимальные высоты для параксиальной и реальной оптимизации 2х гармонических линз

Исходя из рисунка можно увидеть, что найденные оптимальные высоты в ходе параксиального алгоритма и реальной оптимизации (трассировки) отличаются не значительно в рамках размерности (мкм).

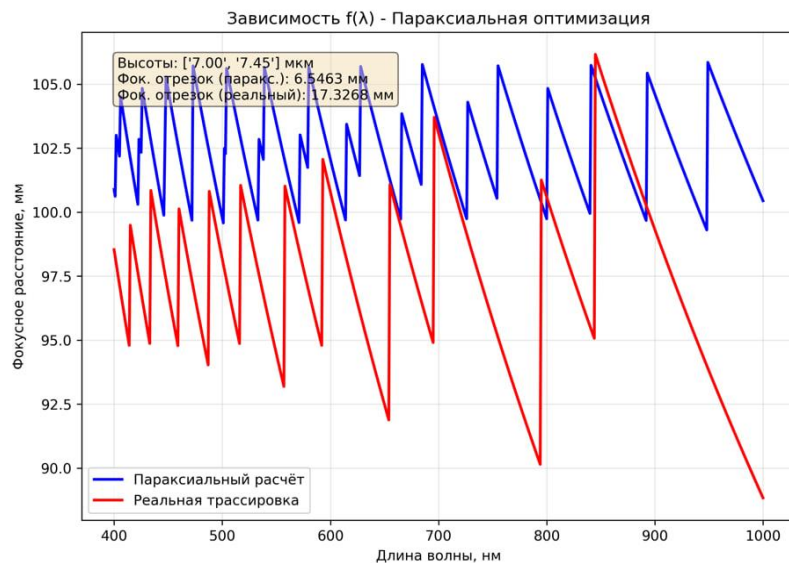


Рисунок 7 – Сравнительный анализ параксиального алгоритма и метода трассировки лучей 2х линз. Значения оптимальных высот соответствуют параксиальному случаю

Если подставить оптимальные высоты при параксиальном приближении, то можно заметить, что величины фокальных отрезков отличаются практически в три раза, что говорит об острой неточности параксиального приближения. При этом величина колебаний фокусного расстояния при трейсинге увеличивается, что говорит об увеличении ширины фокального пятна.

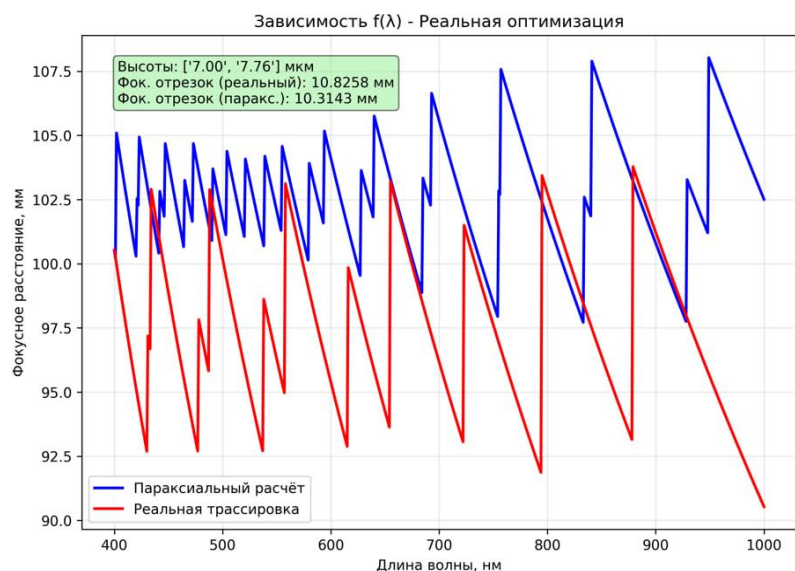


Рисунок 8 - Сравнительный анализ параксиального алгоритма и метода трассировки лучей 2х. Значения оптимальных высот соответствуют реальному случаю

Если аналогично найти оптимальные высоты в реальной оптимизации и подставить их в параксиальный алгоритм, то величина колебаний фокусного расстояния у параксиального алгоритма увеличится, а при трассировки верхний пик колебания практически фиксирован и не поднимается выше.

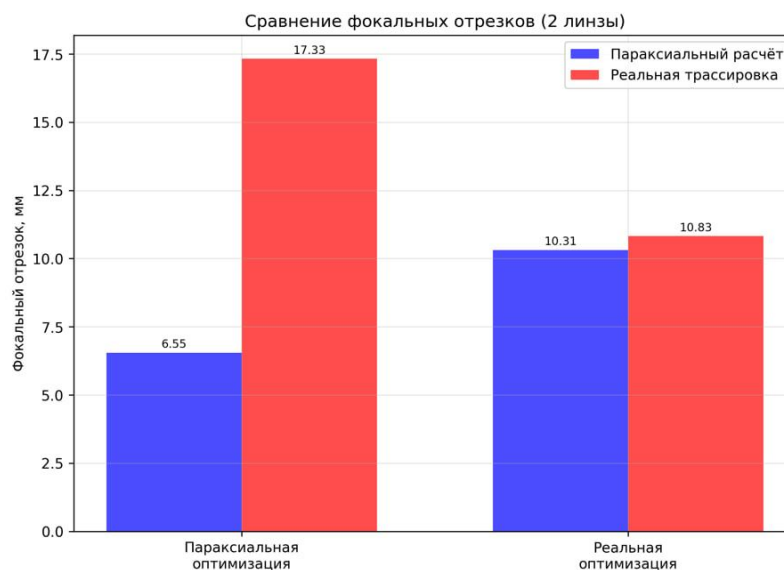


Рисунок 9 – Сравнение фокальных значений для 2х гармонических линз

Сравнительный анализ показал, что найденные оптимальные высоты микрорельефа для параксиального и реального случаев различаются незначительно в абсолютных единицах (мкм).

Однако величины фокальных отрезков отличаются практически в три раза, что свидетельствует о существенной неточности параксиального приближения для оценки реального фокального сдвига.

При использовании трассировки также зафиксировано увеличение ширины фокального пятна за счет учета aberrаций.

4.2.2 Опыт с тремя гармоническими линзами

Данные были взяты из таблицы 2.

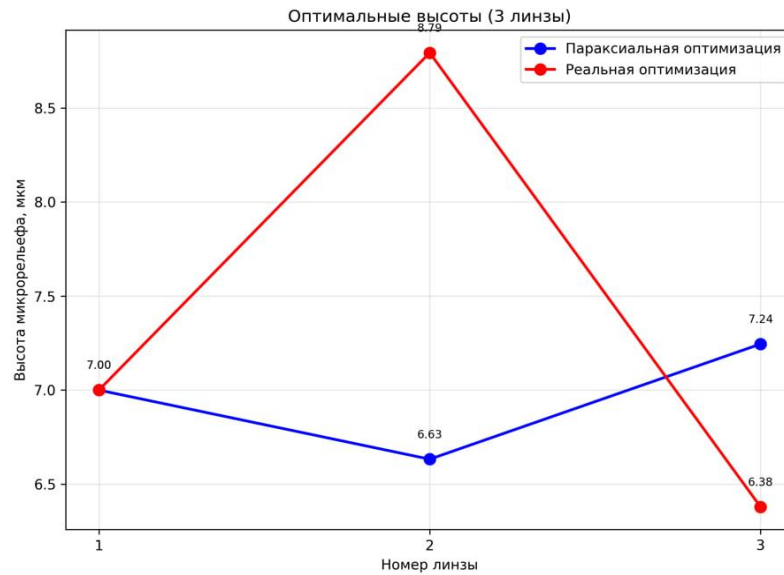


Рисунок 10 – Оптимальные высоты для параксиальной и реальной оптимизации 3х гармонических линз

В ходе алгоритма были найдены оптимальные значения высот, которые соответствуют минимальным значениям фокальных отрезков.

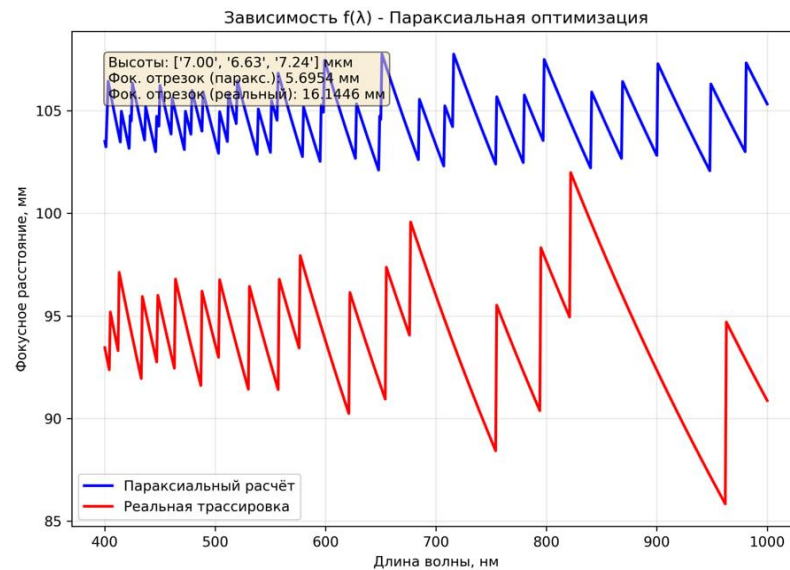


Рисунок 11 – Сравнительный анализ параксиального алгоритма и метода трассировки лучей 3х линз. Значения оптимальных высот соответствуют параксиальному случаю

Если подставить оптимальные высоты при параксиальном приближении, то можно заметить, что величины фокальных отрезков отличаются практически в три раза. При этом величина колебаний фокусного расстояния при трейсинге увеличивается и смещается диапазон колебаний фокусного расстояния.

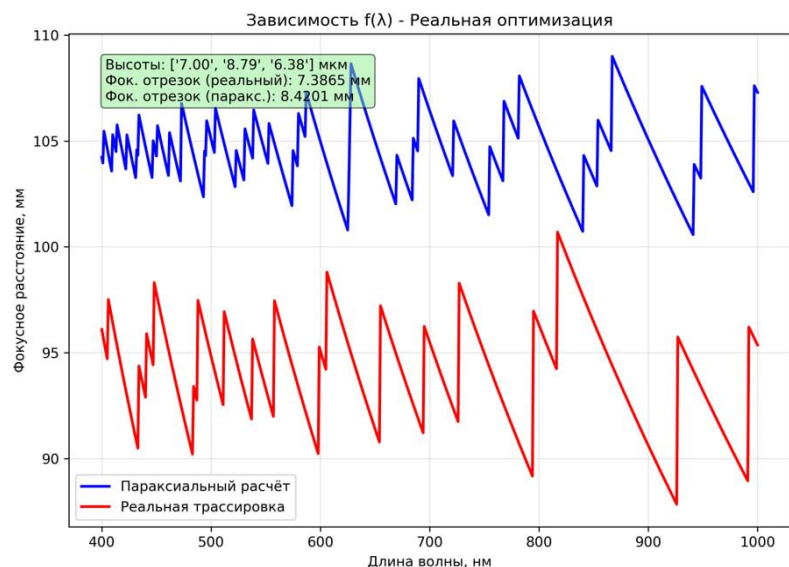


Рисунок 12 – Сравнительный анализ параксиального алгоритма и метода трассировки лучей 3х. Значения оптимальных высот соответствуют реальному случаю

При найденных оптимальных высотах в реальной системе величина фокального пятна уменьшалась примерно на 3 мм. При этом фокальный отрезок в параксиальном приближении примерно на 2 мм шире, чем на рис.11.

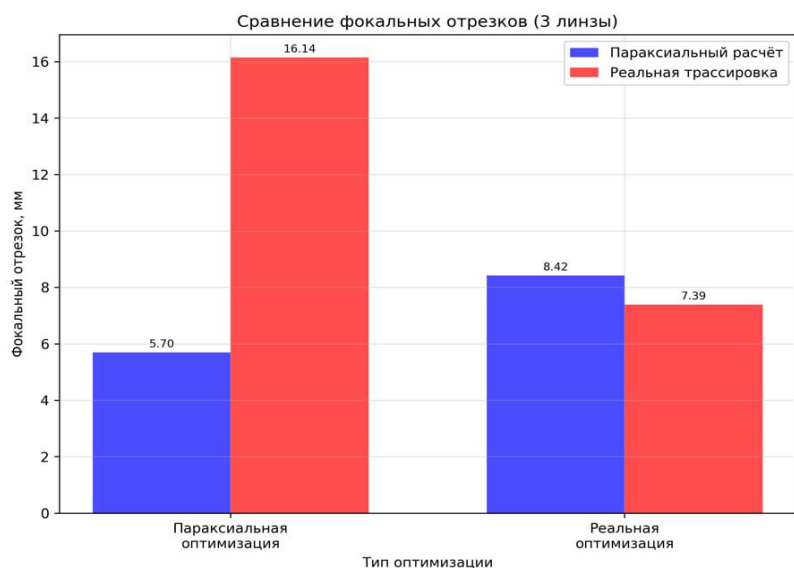


Рисунок 13 – Сравнение фокальных значений для 3х гармонических линз

При анализе трехлинзовой системы выявлено, что параксиальное приближение также занижает величину фокального отрезка в 3 раза.

В ходе реальной трассировки наблюдается не только расширение, но и смещение диапазона колебаний фокусного расстояния.

Применение алгоритма оптимизации в реальной системе позволило уменьшить фокальный отрезок на 3,44 мм, в то время как параксиальная модель предсказывала улучшение менее чем на 1 мм.

4.2.3 Опыт с четырьмя гармоническими линзами

Данные были взяты из таблицы 3.

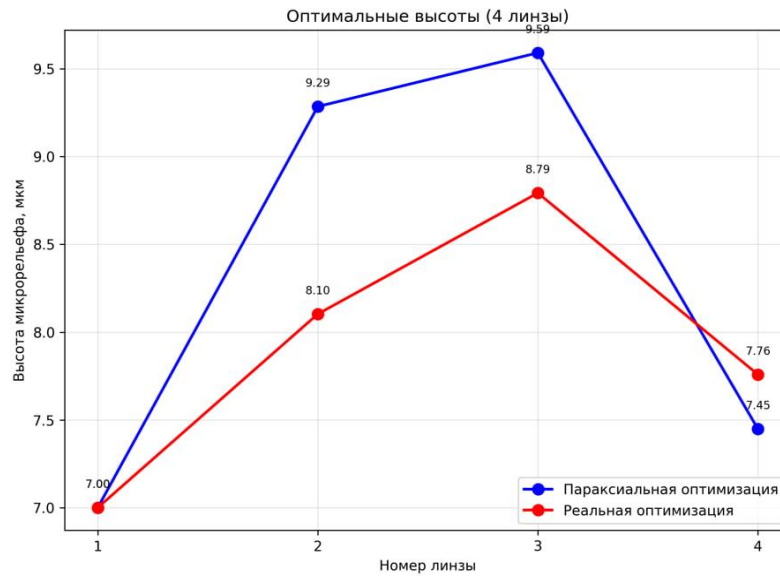


Рисунок 14 – Оптимальные высоты для параксиальной и реальной оптимизации 4х гармонических линз

В ходе алгоритма были найдены оптимальные значения высот, которые соответствуют минимальным значениям фокальных отрезков.

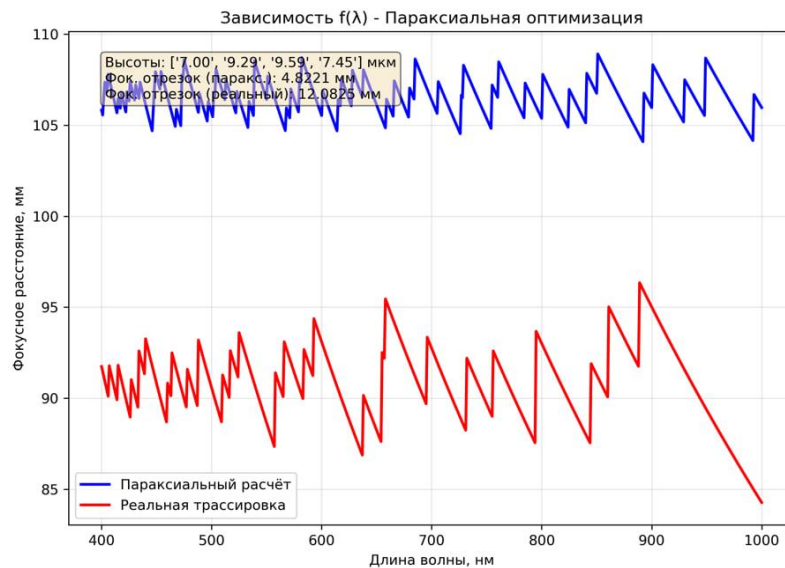


Рисунок 15 – Сравнительный анализ параксиального алгоритма и метода трассировки лучей 4х линз. Значения оптимальных высот соответствуют параксиальному случаю

Если подставить оптимальные высоты при параксиальном приближении, то можно заметить, что величины фокальных отрезков отличаются аналогично практически в три раза. При этом величина колебаний фокусного расстояния при трейсинге увеличивается и смещается диапазон колебаний фокусного расстояния.

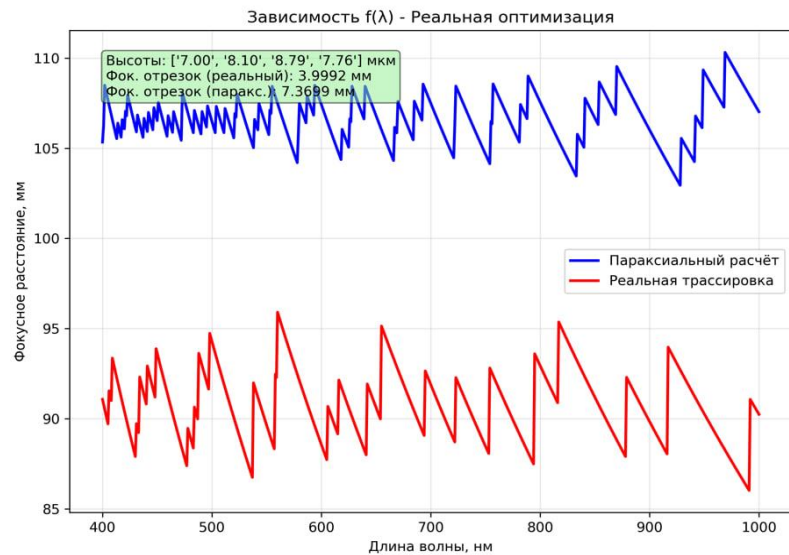


Рисунок 16 – Сравнительный анализ параксиального алгоритма и метода трассировки лучей 4х. Значения оптимальных высот соответствуют реальному случаю

При найденных оптимальных высотах в реальной системе величина фокального пятна уменьшалась примерно на 3 мм. При этом фокальный отрезок в параксиальном приближении примерно на 2,5 мм шире, чем на рис.15.

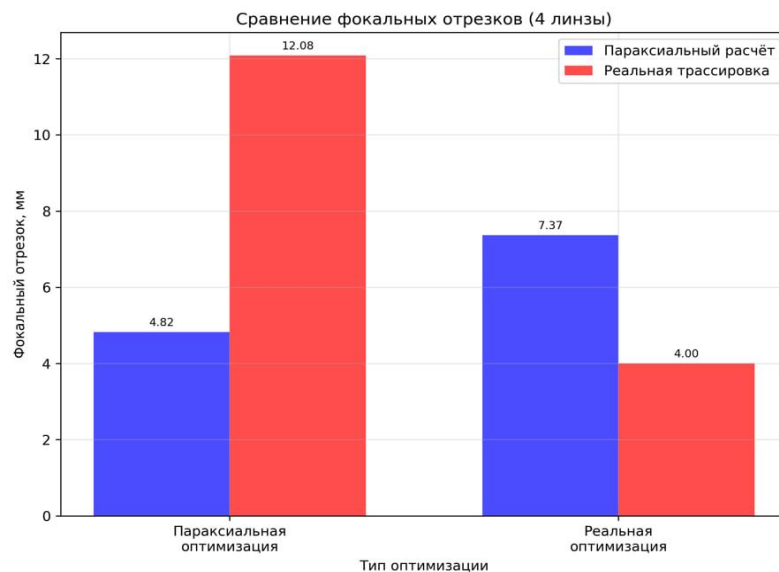


Рисунок 17 – Сравнение фокальных значений для 4х гармонических линз

Для системы из четырех линз (таблица 3) подтвердилась общая закономерность: фокальный отрезок при трассировке в 3 раза шире параксиального.

Трассировка показала, что реальная величина фокального пятна при оптимизации сокращается примерно на 3,4 мм.

Было обнаружено, что при найденных оптимальных высотах верхний пик колебаний фокусного расстояния стабилизируется, что критически важно для качества изображения.

ЗАКЛЮЧЕНИЕ

В итоге прохождения научно-исследовательской работы было проведено моделирование фокального сдвига системы гармонических линз в видимом диапазоне методом трассировки лучей.

Разработан комплексный метод и программное обеспечение для трассировки лучей, учитывающее сферическую аберрацию и дисперсионные свойства стекла ВК7.

Выявлено, что параксиальное приближение обладает острой неточностью, занижая реальную величину фокального сдвига примерно в 3 раза для всех исследованных конфигураций (2, 3 и 4 линзы).

Определены оптимальные соотношения высот микрорельефа. Для системы из 2-х линз результаты совпали с теоретическими данными.

Для систем из 3-х и 4-х линз удалось достичь сокращения длины фокального отрезка в 2.6 и 3.2 раза соответственно.

Доказано, что использование метода трассировки лучей при оптимизации позволяет сократить реальный фокальный отрезок на величину до 3,44 мм, что недоступно при использовании только параксиальных формул.

Таким образом, верифицирована необходимость использования реальной трассировки лучей для прецизионного проектирования систем гармонических линз.

Выступал на МНК 8 апреля 2026 года.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Глазунов, В. А. Геометрическая и волновая оптика: учеб. пособие / В. А. Глазунов. – М.: Лань, 2021. – 320 с.
- 2 Макаров, Ф. Ф. Основы оптики и оптического приборостроения / Ф. Ф. Макаров. – СПб.: Питер, 2019. – 416 с.
- 3 Борн М., Вольф Э. Основы оптики / пер. с англ. – М.: Наука, 1973.
- 4 Расчёт дифракционной линзы с фиксированным положением фокуса при нескольких длинах волн / Л.Л. Досколович, Е.А. Безус, Д.А. Быков, Р.В. Скиданов, Н.Л. Казанский // Компьютерная оптика – 2019 – Т.43, № 43, № 6 – С.946-955. – DOI: 10.18287/2412-6179-2019-43-6-946-955.
- 5 Скиданов, Р.В. Экспериментальное исследование дифракционных линз для работы с излучением нескольких заданных длин волн / Р.В. Скиданов, Л.Л. Досколович, С.В. Ганчевская, В.А. Бланк, В.В. Подлипнов, Н.Л. Казанский // Компьютерная оптика. – 2020. – Т. 44, № 1. – С. 22-28. – DOI: 10.18287/2412-6179-СО-646.
- 6 Системы обобщенных гармонических линз для формирования изображений / Р. В. Скиданов, С. В. Ганчевская, В. С. Васильев, В. А. Бланк // Оптический журнал. – 2022. – Т. 89, № 3. – С. 13-19. – DOI 10.17586/1023-5086-2022-89-03-13-19. – EDN XNITMQ.
- 7 Гармоническая киноформная линза: дифракционная эффективность и хроматизм / Г. И. Грейсух, В. А. Данилов, С. А. Степанов [и др.] // Оптика и спектроскопия. – 2018. – Т. 125, № 2. – С. 223-228. – DOI 10.21883/ОС.2018.08.46364.96-18. – EDN XTLTGH.
- 8 Sweeney D. W. Harmonic diffractive lenses // Appl. Opt. 1995. 34, N 14. P. 2469–2476.
- 9 Харитонов С.И. Геометрооптический расчёт фокального пятна гармонической дифракционной линзы // Компьютерная оптика. 2016. 40, № 3. С. 331–337.
- 10 Хонина С.Н. Анализ фокусировки гармонической дифракционной линзой с учётом дисперсии показателя преломления // Компьютерная оптика. 2017. 41, №3. С. 338–347.
- 11 Bergstra J., Bengio Y. Random search for hyper-parameter optimization // Journal of Machine Learning Research. 2012. Vol. 12. P. 281-305.
- 12 Hutter F., Kotthoff L., Vanschoren J. Automated Machine Learning: Methods, Systems, Challenges. Springer, 2019. 129 p.
- 13 Liashchynskiy P., Liashchynskiy P. Grid search, random search, genetic algorithm: A big comparison for NAS [Electronic resource] // arXiv preprint. 2019. URL: <https://arxiv.org/abs/1912.06059>.

- 14 Ruder S. An overview of gradient descent optimization algorithms [Electronic resource] // arXiv preprint. 2016. URL: <https://arxiv.org/abs/1609.04747>.
- 15 Haupt R.L., Haupt S.E. Practical Genetic Algorithms. 2nd ed. John Wiley & Sons, 2004. 253p.
- 16 Eiben A.E., Smith J.E. Introduction to Evolutionary Computing. Springer, 2015. 287 p.

ПРИЛОЖЕНИЕ А

Базовый класс для трассировки лучей

```
class HarmLens:
```

```
    """Класс для моделирования гармонических (дифракционных) линз"""
```

```
    def __init__(self, N=10000):
```

```
        self.N = N # Количество точек в профиле
```

```
        # Параметры луча
```

```
        self.x0 = self.y0 = self.z0 = 0.0
```

```
        self.ax = self.ay = self.az = 0.0
```

```
        self.xL = self.yL = self.zL = 0.0
```

```
        self.axL = self.ayL = self.azL = 0.0
```

```
        self.Nx = self.Ny = self.Nz = 0.0
```

```
        self.eps1 = self.eps2 = self.G = 0.0
```

```
        self.II = 0
```

```
        self.rL = 0.0
```

```
        # Параметры линзы
```

```
        self.P = 100.0 # Параметр кривизны (радиус для сферической, фокус для  
параболической)
```

```
        self.Diameter = 10.0 # Диаметр линзы (мм)
```

```
        self.Thickness = 2.0 # Толщина линзы в центре (мм)
```

```
        self.n1 = 1.0 # Показатель преломления среды до линзы
```

```
        self.n2 = 1.5 # Показатель преломления материала линзы
```

```
        self.Harm = 0 # Тип линзы: 0-обычная, 1-гармоническая
```

```
        self.Hr = 0.001 # Высота рельефа для гармонической линзы (мм)
```

```
        self.Lw = 0.000632 # Длина волны (мм) - 632 нм
```

```
        self.Lw0 = 0.000532 # Базовая длина волны для расчётов (мм) - 532 нм
```

```
        self.Z_Lens = 0.0 # Положение линзы на оси Z (мм)
```

```
        self.Type = 2 # Тип поверхности: 0-плоская, 1-сферическая, 2-параболиче-  
ская
```

```
        # Массивы профиля
```

```
        self.x = np.linspace(0, self.Diameter/2, self.N)
```



```

self.Section = np.zeros(self.N)
self.Zones = np.zeros(self.N, dtype=int)

# Дисперсионные данные
self.n_1 = np.ones(1000) * self.n1
self.n_2 = np.ones(1000) * self.n2

# Создаём профиль по умолчанию
self.create_profile()

def create_profile(self):
    """Создание профиля поверхности в зависимости от типа"""
    if self.Type == 0:    # Плоская
        self.plane()
    elif self.Type == 1:  # Сферическая
        self.S_Lens(0, self.n1, self.n2)
    elif self.Type == 2:  # Параболическая
        self.P_Lens(0, self.n1, self.n2)

    # Если гармоническая линза - преобразуем
    if self.Harm > 0:
        self.harmonization(self.Hr)

def P_Lens(self, l, n1, n2):
    """Создание параболического профиля"""
    # Защита от деления на ноль
    if abs(self.P) < 1e-10:
        self.P = 1e10 # Очень большое значение для почти плоской поверхности

    for i in range(self.N):
        x_val = self.Diameter / 2 / self.N * i
        self.x[i] = x_val
        # Параболическая формула:  $z = \text{Thickness} - x^2/(2P)$ 
        self.Section[i] = self.Thickness - x_val * x_val / (2.0 * self.P)
        if self.Section[i] < 0:

```

```

        self.Section[i] = 0
    return 1

def S_Lens(self, l, n1, n2):
    """Создание сферического профиля"""
    RL = self.P

    # Защита от деления на ноль и некорректных значений
    if abs(RL) < 1e-10:
        RL = 1e10 # Очень большой радиус = почти плоская

    kf = 1 if RL > 0 else -1

    for i in range(self.N):
        x_val = self.Diameter / 2 / self.N * i
        self.x[i] = x_val
        H = RL * RL - x_val * x_val

        if H > 0:
            self.Section[i] = self.Thickness + kf * (math.sqrt(H) - abs(RL))
        else:
            self.Section[i] = 0

        if self.Section[i] < 0:
            self.Section[i] = 0
    return 1

def plane(self):
    """Создание плоского профиля"""
    for i in range(self.N):
        self.x[i] = self.Diameter / 2 / self.N * i
        self.Section[i] = self.Thickness
    return 1

def harmonization(self, h):

```

```

        """Преобразование профиля в гармонический (создание зонной пластинки)"""
        if abs(h) < 1e-10: # Защита от нулевой высоты
            h = 1e-6

        for i in range(self.N):
            if abs(h) > 1e-10: # Проверка деления на ноль
                hd = int(self.Section[i] / h)
                self.Section[i] = self.Section[i] - hd * h
        return 1

    @staticmethod
    def harm_lam(m, lam0, lamt):
        """Вычисление гармонического коэффициента для длины волны"""
        if abs(lam0) < 1e-10 or abs(lamt) < 1e-10:
            return 1.0

        li = int(m * lam0 / lamt)
        lf = m * lam0 / lamt

        if abs(lf - li) < 0.5:
            lam_i = lam0 * m / li
        else:
            lam_i = lam0 * m / (li + 1)

        if abs(lam_i) > 1e-10:
            harm_k = lamt / lam_i
        else:
            harm_k = 1.0

        return harm_k

    def vec(self):
        """Расчёт Z-компоненты направляющего вектора"""
        sum_sq = self.ax*self.ax + self.ay*self.ay
        if 1 - sum_sq > 0:

```

```

        self.az = math.sqrt(1 - sum_sq)
    else:
        self.az = 0
    return 1

def norm(self, xi, yi):
    """Вычисление вектора нормали в точке (xi, yi)"""
    dr = self.Diameter / self.N / 2

    # Защита от деления на ноль
    if abs(dr) < 1e-10:
        dr = 1e-6

    rad = math.sqrt(xi*xi + yi*yi)

    if rad < 1e-10: # В центре
        self.Nx = 0
        self.Ny = 0
        self.Nz = 1
        return 0

    ri = rad / dr
    rd = min(int(ri), self.N-2)

    df = (self.Section[rd+1] - self.Section[rd]) / dr
    dx = df * xi / rad
    dy = df * yi / rad

    delt = math.sqrt(dx*dx + dy*dy + 1)
    if delt > 0:
        self.Nx = -dx / delt
        self.Ny = -dy / delt
        self.Nz = 1 / delt

    return rd

```

```

def norm_m(self, xi, yi, m):
    """Вычисление нормали с масштабированием для гармонических линз"""
    dr = self.Diameter / self.N / 2

    # Защита от деления на ноль
    if abs(dr) < 1e-10:
        dr = 1e-6

    rad = math.sqrt(xi*xi + yi*yi)

    if rad < 1e-10:
        self.Nx = 0
        self.Ny = 0
        self.Nz = 1
        return 0

    ri = rad / dr
    rd = min(int(ri), self.N-2)

    df = m * (self.Section[rd+1] - self.Section[rd]) / dr
    dx = df * xi / rad
    dy = df * yi / rad

    delt = math.sqrt(dx*dx + dy*dy + 1)
    if delt > 0:
        self.Nx = -dx / delt
        self.Ny = -dy / delt
        self.Nz = 1 / delt

    return rd

def beam_lens(self):
    """Основной метод трассировки луча через линзу"""
    # 1. Проверка направляющего вектора

```

```

self.vec()
if abs(self.az) < 1e-10:
    return 0

# 2. Параметрическое уравнение луча
zL = self.Z_Lens

# Защита от деления на ноль
if abs(self.az) < 1e-10:
    return 0

par = (zL - self.z0) / self.az

# 3. Начальное приближение
xL = self.x0 + par * self.ax
yL = self.y0 + par * self.ay
self.zL = zL

# 4. Бинарный поиск точки пересечения
t0 = 0.0
tL = par

# Защита от некорректного dx
if len(self.x) > 1:
    dx = self.x[1] - self.x[0]
    if abs(dx) < 1e-10:
        dx = 1e-6
    else:
        dx = 1.0

rL = math.sqrt(xL*xL + yL*yL)

if rL < self.Diameter/2 and abs(par) > 1e-10:
    dt = tL - t0
    iterations = 0

```

```

while abs(dt) > 1e-9 and iterations < 100:
    tM = (t0 + tL) / 2
    xL = self.x0 + tM * self.ax
    yL = self.y0 + tM * self.ay
    zL = self.z0 + tM * self.az
    rL = math.sqrt(xL*xL + yL*yL)

    i = min(int(rL / dx), self.N-1) if dx > 0 else 0

    if zL - self.Z_Lens < self.Section[i]:
        tL = tM
    else:
        t0 = tM

    dt = tL - t0
    iterations += 1

# 5. Сохраняем точку пересечения
self.xL = xL
self.yL = yL
self.zL = zL
self.rL = rL
self.II = i

# 6. Вычисляем нормаль
if self.Harm == 1:
    if abs(self.Lw0) > 1e-10:
        self.norm_m(xL, yL, self.Lw / self.Lw0)
    else:
        self.norm(xL, yL)
elif self.Harm > 1:
    kHarm = self.harm_lam(self.Harm, self.Lw0, self.Lw)
    self.norm_m(xL, yL, kHarm)
else:

```

```

        self.norm(xL, yL)

# 7. Закон Снеллиуса
s = self.ax*self.Nx + self.ay*self.Ny + self.az*self.Nz
if s <= 1:
    self.eps1 = math.acos(s)
else:
    self.eps1 = math.pi / 2

# 8. Показатели преломления
n1_val = self.n1
n2_val = self.n2

# 9. Угол преломления
sin_eps2 = n1_val / n2_val * math.sin(self.eps1)
if abs(sin_eps2) < 1:
    self.eps2 = math.asin(sin_eps2)
else:
    self.eps2 = math.asin(-math.sin(self.eps1))

# 10. Новый направляющий вектор
self.G = n2_val * math.cos(self.eps2) - n1_val * math.cos(self.eps1)
self.axL = n1_val * self.ax / n2_val - self.Nx * self.G / n2_val
self.ayL = n1_val * self.ay / n2_val - self.Ny * self.G / n2_val

az_sq = 1 - self.axL*self.axL - self.ayL*self.ayL
if az_sq > 0:
    self.azL = math.sqrt(az_sq)
else:
    self.azL = 0

return 1
else:
    return 0

```



```

def beam_plane_r(self, ZR):
    """Продолжение луча до плоскости регистрации"""
    if abs(self.azL) > 1e-10:
        t = (ZR - self.zL) / self.azL
        xR = self.xL + t * self.axL
        yR = self.yL + t * self.ayL
        return xR, yR
    return None, None

def set_parameters(self, **kwargs):
    """Установка параметров линзы с последующим обновлением профиля"""
    for key, value in kwargs.items():
        if hasattr(self, key):
            setattr(self, key, value)

    # Обновляем профиль после изменения параметров
    self.create_profile()

```